

Let's now explore a few options...

Note: all options discussed in textbook will not be covered in class!

Option: strict alternation *[sw]*

- ◆ Threads/processes take turns executing their critical regions
 - ◆ *turn == 0* $\rightarrow$ thread/process 0 executes
 - ◆ *turn == 1* $\rightarrow$ thread/process 1 executes

Thread/Process 0

```
while(1) {  
    while(turn != 0);  
    critical_region_0();  
    turn = 1;  
    non_critical_region_0();  
}
```

Thread/Process 1

```
while(1) {  
    while(turn != 1);  
    critical_region_1();  
    turn = 0;  
    non_critical_region_1();  
}
```


Option: strict alternation

◆ *Pros*

- ◆ Mutual exclusion *correctly* maintained

◆ *Cons*

- ◆ *Busy waiting* required
- ◆ Thread/process forced to *wait* even if other thread/process executing its non critical region
 - ◆ Violates one of the requirements we looked at earlier!
 - *Makes method unsuitable for use in practice*
 - ◆ Gets worse as num competing threads/processes increases

Processor cycles
unnecessarily wasted!